Chapter05

EDU-100 Agent PM 실무 트레이닝

W5. 검수 자동화

시나리오 자동 테스트부터 산출물 현행화까지

AP-Framework V0.43 · 로컬 → 배포 2단계 검증 · 테스트 코드 0줄 · 강의:실습 15:85

로디 박한근

About



로디 박한근

1st Agent PM
25년 PM/기획/컨설팅 경력
100개+ 프로젝트 수행
로디 서비스 & 디자인 아카데미 대표
Agent PM 전자책 시리즈 저자

phk4568@naver.com

Chapter05.
W5 검수 자동화 —
시나리오 자동 테스트부터 산출물 현행화까지

1. 수업 개요

지난 주(W4) 한눈에 요약

App Router 풀스택 src/frontend · app/api Route Handler + Supabase Auth → 오늘 이 코드를 검증한다

Vercel 라이브 배포 살아있는 URL 1개 이상 확보 → 오늘 배포본을 자동 테스트한다

AI 생성 코드 빠르게 만들어졌지만 검증되지 않았다 → 오늘 시나리오로 검수한다

설계 산출물 구현하며 사양이 바뀌었다 → 오늘 코드 기준으로 현행화한다

W4 = 만들었다 → W5 = 만든 것이 기대대로 도는지 증명하고, 문서를 현실에 맞춘다

기본 개념 Check — Phase 3 Gate · 사전 과제

1	W4 과제 완료 — App Router 코드 + Supabase Auth + 라이브 배포 1회 이상
2	저장소 클론 + npm install 완료 (src/frontend)
3	.env.local 5종 등록 — Supabase URL/ANON/SERVICE_ROLE · JWT_SECRET · SITE_URL
4	auto-scen-test 설치 — bash setup.sh (Playwright + Chromium + openpyxl + pandas)
5	n8n Cloud 가입 + Slack Incoming Webhook 생성 (실습 6용)

1~4는 부록 B.1~B.2.1 사전 과제 · 실습 1(15분)은 '전원이 실제로 기동되는가'를 점검하는 시간

오늘의 흐름 — 9단계 파이프라인

실습 1·2 로컬 Dev 환경 셋팅(localhost:3000) → 테스트 시드 10건 (계정 4 + 콘텐츠 6)

실습 3 ★핵심 — 로컬에서 시나리오 자동 테스트 (P-501 + /xlsx + P-507)

실습 4 결과 판독 → 오탐/실제 버그 분류 → 시나리오·코드 수정 → 재실행 (Pass Rate 80%)

실습 5·6 배포본 재실행 + 라이브 육안 검증(P-505) → n8n + Slack 결과 자동 공유

실습 7·8·9 산출물 현행화(P-508) → 완료보고서(P-504+P-208, 17/17) → 발표 · Gate 인증 · 수료

테스트 코드는 한 줄도 작성하지 않는다 — PM은 '무엇을 검증할지'를 시나리오로 정의한다

3막으로 보면

1막 · 실습 1~4	로컬에서 검증한다 — 환경·시드 → 자동 판정 → 오탐/버그 분류 → Pass Rate 80%
2막 · 실습 5~6	배포본에서 다시 검증하고 공유한다 — --url 교체 재실행 + P-505 + Slack 자동 전송
3막 · 실습 7~9	현행화·마무리·수료 — 코드→설계 역반영 → 완료보고서 17/17 → Phase 4 Gate 인증

로컬과 배포는 대체가 아니라 순서다 — 서로 다른 결함을 잡는다

5주차 = 마지막 주 · 오늘을 끝내면 .progress.md 100% = 수료

강의 1. 테스트 전략 · 로컬→배포 2단계 검증 · 산출물 현행화

왜 테스트 코드를 직접 작성하지 않는가

개발자의 단위 테스트	PM의 시나리오 자동 테스트
Jest / Supertest 코드를 직접 작성	엑셀에 '무엇을 확인할지'만 작성
코드가 늘면 테스트도 유지보수 대상	Playwright가 실제 화면을 클릭·입력
비개발 PM에게 진입장벽이 높다	코드 0줄 — 스킬이 실행·판정
화면이 아니라 함수 단위를 검증	증적(스크린샷+XLSX) 자동 수집

PM이 가장 잘하는 일에 집중한다 — 무엇을 검증할지 정의하고, 실행은 AI가 맡는다

입력 프롬프트 "기능명세서.md와 API스펙.md를 읽고 테스트시나리오.md를 정밀화해줘"

검증의 2축 — 환경 축 · 방식 축

환경 축 ①	로컬 Dev (localhost:3000) — 빠른 반복, 셀렉터·판정 룰 튜닝에 최적
환경 축 ②	배포본 (Vercel) — 빌드 · 서버리스 · HTTPS · 캐시, 진짜 환경의 결함
방식 축 ①	/auto-scen-test — 시나리오대로 UI 자동 구동 (Playwright, 증적 자동)
방식 축 ②	P-505 라이브 육안 검증 — 레이아웃 · 에셋 · 환경 의존 이슈

코드 작성 0줄 · 두 축이 서로 다른 결함을 잡는다

시나리오 자동 테스트의 동작 원리

1. 입력 시나리오 엑셀 — id / 사전조건 / 수행절차(DSL) / 기대 결과 / 검증 포인트

2. 실행 케이스별 DSL 실행 — goto → fill → click → wait (Playwright가 Chromium 조종)

3. 수집 판정 영역 텍스트 추출 + 전체 페이지 스크린샷 캡처

4. 판정 기대 결과 기준 Pass / Fail / N-A / Block 자동 판정

5. 기록 결과 XLSX — 테스트결과 · 실행일시 · 응답요약 · 증적이미지 + 결과 요약 시트

openpyxl이 엑셀을 읽고 → Playwright가 Chromium을 열어 실행 → 다시 openpyxl로 결과 기록

판정 4종과 대응 — 누가 고치는가

Pass	기대값이 실제로 검출됨 — 조치 없음
Fail (오탐)	기대 문구가 실제 화면과 다름 → 시나리오 수정 (PM)
Fail (실버그)	기능이 실제로 동작하지 않음 → 코드 수정 후 재실행 (개발 / Claude)
N/A	자동 판정 근거 없음 → 스크린샷 수동 확인 · 기대값 명시로 전환 (PM)
Block	실행 자체가 불가 → 셀렉터 · 수행절차 · 데이터 문제 (PM)

판정 근거가 없으면 Pass가 아니라 N/A — 룰을 느슨하게 만들면 증적이 아니라 알리바이가 된다

스킬을 부른다 — 명령어를 외우지 않는다

권장 — 스킬 호출 (자연어)	참고 — CLI 직접 실행
"이 엑셀로 로컬 자동 테스트 돌려줘"	run_tests.py --map-check ...
"3건만 먼저 스모크로"	run_tests.py --discover --url ...
"TC-018만 다시 돌려줘"	run_tests.py ... --limit 3
"이번엔 배포본에 같은 시나리오로"	run_tests.py ... --only TC-018

테스트 코드 0줄 — 그런데 CLI 옵션을 외우게 하면 부담을 다른 곳으로 옮긴 것뿐이다

교안의 python3 run_tests.py 블록 = 스킬이 내부적으로 실행하는 명령(결과 해석용 참고 자료)

실습 1 · 2. 로컬 Dev 환경 셋팅 + 시드 데이터

실습 1 — 로컬 Dev 테스트 환경 셋팅 (15분)

Step 1 (6분) 확인 → 판단 → 기동 — lsof -i :3000 / curl로 중복 실행부터 확인 후 npm run dev

Step 2 (5분) DB 연결 확인 — Supabase REST 조회로 응답이 오는지 점검

Step 3 (4분) auto-scen-test 사용 가능 상태 점검 — 스킬에 "점검해줘" 한 줄 (도구 설치 + 셀렉터 후보)

환경변수(.env.local)는 서버 기동 시 1회만 읽힌다 — 값을 바꿨다면 반드시 재기동

가장 흔한 사고: 이미 떠 있는데 또 띄우다 EADDRINUSE — 확인이 먼저다

입력 프롬프트 ① "npm run dev로 로컬 서버를 띄우고 200 응답과 .env.local 인식 여부를 확인해줘"
입력 프롬프트 ② "/auto-scen-test 스킬이 쓸 수 있는 상태인지 점검하고 로그인 화면 셀렉터 후보를 뽑아줘"

로컬 서버 명령어 — 확인 → 종료 → 기동 → 검증

macOS / Linux

```
# ① 확인 — 포트 점유 프로세스(PID)
lsof -i :3000
# ① 확인 / ④ 검증 — 응답 코드
#   200 정상 · 000 꺼짐 · 500 서버오류
curl -s -o /dev/null -w "%{http_code}\n" \
  http://localhost:3000/

# ② 종료 — 서버 돌던 터미널에서
Ctrl + C
# ② 종료 — PID 지정 (안 죽으면 -9)
kill 54321   →   kill -9 54321
# ② 종료 — 한 줄 정리 (마지막 수단)
kill -9 $(lsof -t -i :3000)

# ③ 기동  (포트 변경: -- -p 3001)
cd <프로젝트>/src/frontend  &&  npm run dev
```

Windows (PowerShell)

```
# ① 확인 — 맨 끝 숫자가 PID
netstat -ano | findstr :3000

# ② 종료
taskkill /PID 54321 /F

# ③ 기동
cd <프로젝트>\src\frontend
npm run dev

# ④ 검증
curl.exe -s -o NUL -w "%{http_code}`n" \
  http://localhost:3000/
```

kill → 안 죽으면 kill -9 · .env.local을 고쳤다면 반드시 ②→③ 재기동

setup.sh가 설치하는 4종 — 무엇이 무엇인가

Playwright

브라우저 자동 조작 도구

- 사람 대신 브라우저를 조종
- 클릭 · 입력 · 스크린샷 수행
- 자동 테스트의 실행 엔진
- Python 라이브러리

Chromium

크롬의 오픈소스 원본

- Playwright가 조종할 브라우저 본체
- 없으면 열 창이 없다
- 용량 수백 MB
- 사내망에서 다운로드 지연 주의

openpyxl / pandas

엑셀 읽기·쓰기 / 집계

- 시나리오 엑셀을 읽어 들임
- 판정 결과·증적을 기록
- 결과 요약 시트 집계
- 표 데이터 처리

openpyxl이 읽고 → Playwright가 Chromium을 열어 실행 → 다시 openpyxl로 기록

실습 2 — 테스트 시드 데이터 10건 (20분)

Step 1 (5분) 시나리오가 요구하는 데이터 식별 — 계정 등급 · 콘텐츠 상태 · DB 현황 대조

Step 2 (10분) 계정 4건(무료·유료·만료·관리자) + 콘텐츠 6건(공개·잠금·임시저장·검색) 생성

Step 3 (5분) 적용 및 검증 — REST 조회 + 브라우저에서 실제 로그인까지 확인

정리 스크립트 실습 종료 후 SQL Editor에서 실행 — 확인 SELECT → 삭제(자식→부모) → 재확인

적용·정리 모두 Supabase 대시보드 SQL Editor에서 · DELETE 전에 SELECT로 대상 건수부터 확인

시드가 없으면 Fail이 아니라 Block — 로컬·배포가 같은 DB면 조건 없는 DELETE는 운영 데이터를 지운다

입력 프롬프트 "테스트시나리오.md를 읽고 필요한 계정·콘텐츠 시드 10건 생성 SQL과 정리 스크립트를 만들어줘"

실습 3 · 4. 로컬 시나리오 자동 테스트 + 결과 판독

실행용 엑셀 필수 사양 — 지난 기수 최대 시간 손실 지점

시트	케이스 표 단일 시트 — 개요·표지 시트가 앞에 있으면 '필수 컬럼 매핑 실패: id'
헤더	1행 고정 · 병합셀·부제목행 금지 — 헤더 오탐 시 컬럼 전체 미인식
TC-ID (필수)	빈 값·중복 금지 — ID가 빈 행은 로더가 조용히 건너뛴다(케이스 누락)
수행절차 (필수)	DSL 문법 전용 — 자연어면 전 케이스 Block
사전조건 · 대상 URL	로그인 DSL / 화면 경로 — 없으면 로그인 케이스가 Block
기대 결과 · 검증 포인트	화면 실제 문구를 따옴표로 — 없으면 Pass가 아니라 N/A

헤더 7종을 그대로 쓰고 TC-ID를 모두 채운다 — 실행 후 케이스 수를 시나리오와 대조

입력 예시 "/xlsx ... 시트 1개, 헤더는 TC-ID·수행절차·사전조건·대상 URL·기대 결과로, 절차는 DSL로 만들어줘"

수행절차 DSL 7개 동사 — 자연어면 전 케이스 Block

이동 · 입력	동작 · 대기
goto /login — 페이지 이동	click 로그인 — 버튼 클릭
fill #email free@test.io — 입력창 채우기	wait 2s / wait 대시보드 — 대기
select #plan Pro — 드롭다운 선택	waitgone .loading — 요소 소멸 대기
press Enter — 키 입력	대상과 값은 2칸 이상 공백으로 구분

판정 영역(result_container)을 특정하지 못하면 페이지 전체로 폴백 — 이 영역 보강이 판정 정확도의 핵심

실습 3 — 로컬 시나리오 자동 테스트 (40분) ★핵심

Step 1 (12분) P-501 정밀화 → /xlsx 변환 — 헤더 7종·단일 시트·DSL 형식을 프롬프트에 명시 (15/17)

Step 2 (10분) 스킬에 컬럼 매핑 확인 요청 + 수행절차를 DSL 문법으로 작성 (자연어면 전 케이스 Block)

Step 3 (8분) 스모크 3건 — "3건만 먼저" 지시로 셀렉터 · 대기시간 · 판정 룰 검증

Step 4 (10분) 전체 실행 → 결과 XLSX + 스크린샷 증적 + Pass/Fail/N-A/Block 집계 자동 보고

로더는 첫 시트만 읽는다 — 다중 시트 산출물을 그대로 넘기면 '필수 컬럼 매핑 실패: id'로 죽는다

입력 프롬프트 ① "테스트시나리오.md에서 UI 검증 가능한 케이스만 골라 /xlsx 단일 시트 엑셀로 만들어줘. 헤더는 TC-ID·수행절차·사전조건·대상 URL·기대 결과로, 절차는 DSL로 만들어줘 "

입력 프롬프트 ② "/auto-scen-test 이 엑셀로 localhost:3000 자동 테스트하되, 먼저 3건만 스모크로 돌려줘"

실습 4 — 결과 판독 및 수정사항 반영 (25분)

Step 1 (5분) 결과 요약 시트부터 — 판정별 건수 · Fail/Block이 특정 화면에 몰렸는지

Step 2 (8분) Fail을 오탐과 실제 버그로 분류 — 근거는 스크린샷과 응답 요약

Step 3 (8분) 오탐은 시나리오를, 버그는 코드를 수정 — 반대로 하지 않는다

Step 4 (4분) 재실행 → Pass Rate 80% 이상이면 로컬 단계 합격

이 실습이 검수의 진짜 알맹이 — 돌리는 건 명령 한 줄, PM의 판단은 'Fail을 어떻게 처리할 것인가'

입력 프롬프트 ① "자동테스트결과_local.xlsx를 읽고 오탐 의심과 실제 버그 의심을 나눠 제시해줘"
입력 프롬프트 ② "/auto-scen-test TC-018 기대 문구를 고쳤어. TC-018만 다시 돌려줘"

실습 5 · 6. 배포본 검증 + n8n · Slack 자동 공유

실습 5 — 배포본 테스트 + 라이브 검증 (25분)

Step 1 (5분) 수정사항 재배포 — git push → Vercel 자동 배포, 빌드 성공 여부 확인

Step 2 (8분) 동일 시나리오를 --url만 바꿔 배포본에 재실행 (결과는 _prod로 분리 저장)

Step 3 (6분) 로컬 vs 배포 차이 분석 — 배포 특이 결함 후보 식별

Step 4 (6분) P-505 라이브 육안 검증 6항목 — 가입·세션·에셋·레이아웃·API·환경변수

시나리오 파일은 그대로 두고 --url만 바꾼다 — 시나리오 기반 테스트의 가장 큰 실무 이점

입력 프롬프트 "/auto-scen-test 같은 시나리오를 이번엔 배포본 (https://내앱.vercel.app)에 돌리고, 로컬 결과와 차이를 분석해줘"

실습 6 — n8n + Slack 결과 자동 공유 (20분)

Step 1 (3분) n8n Cloud 가입 — 사전 과제 권장

Step 2 (4분) Slack Incoming Webhook 생성 → Webhook URL 복사 (웹훅 1개 = 채널 1개)

Step 3 (8분) 06.n8n/n8n_scenario-checkup-cloud.json Import → HTTP 노드 URL 교체 → Active

Step 4 (5분) 폼에 대상 URL + 테스트시나리오.md 제출 → Slack 케이스별 PASS/FAIL + 요약 확인

@n8n/n8n_scenario-checkup-cloud.json 파일에서 실행이 가능하도록, @04.검수문서/테스트시나리오.md 파일을 0.2 버전으로 만들어줘.

검수를 '돌리는 것'을 넘어 '팀이 즉시 보는 것' — 케이스 추가는 코드가 아니라 표에 한 줄

실습 7 · 8 · 9. 산출물 현행화 · 완료보고서 · 수료

실습 7 — P-508 산출물 현행화 (25분)

Step 1 (4분) 코드 ↔ 산출물 차이(Drift) 식별 — app/api · 라우트 · 기능 · 화면 대조

Step 2 (8분) /xlsx — 기능명세서 · 정보구조도 · API명세를 엑셀로 현행화 (변경/추가/삭제 이력)

Step 3 (5분) /sb-creator — 화면설계서 HTML 생성 → 편집 가능 PPTX (보고 · 발표용)

Step 4 (6분) /sb-html2figma — 같은 HTML → scene.json → Figma 네이티브 프레임 (인수인계용)

코드가 진실, 문서가 코드를 따라간다 — 스크린샷을 던지는 것과 편집 가능한 프레임을 넘기는 것은 다르다

입력 프롬프트 "현행 App Router 코드를 기준으로 기능명세·정보구조·API명세 엑셀을 갱신해줘"

실습 8 — P-504 + P-208 (20분) · Document Chaining 17/17

Step 1 (8분) P-504 테스트결과보고서 — 로컬·배포 2종 + P-505 라이브 검증 종합 (#16)

Step 2 (9분) P-208 완료보고서 — 착수보고서 계획 대비 실적 · 교훈 · 향후 계획 (#17)

Step 3 (3분) 커밋 · push + .progress.md 100% 갱신 (Phase 4 통과)

예시 실적: 로컬 42건 85.7% → 배포 90.5%(핫픽스 후) · 라이브 검증 6/6 통과 · 전체 판정 Pass

입력 프롬프트 ① "자동테스트결과 2종과 배포검증결과를 종합해 테스트결과보고서를 작성해줘"
입력 프롬프트 ② "착수보고서 계획 대비 최종 실적을 정리해 완료보고서를 작성해줘"

실습 9 — 발표 · Phase 4 Gate 인증 · 수료 (40분)

(a)	App Router 코드 동작
(b)	라이브 배포 URL 1개 이상
(c)	로컬 Dev 자동 테스트 Pass Rate 80% 이상
(d)	배포본 자동 테스트 재실행 + P-505 라이브 검증 통과
(e)	산출물 현행화 — 설계 ↔ 코드 정합
(f)	완료보고서 작성

여섯 항목을 모두 충족하면 .progress.md 100% = Phase 4 Gate Pass = 수료

W5 종료 시 남는 산출물

#15	테스트시나리오.md (P-501, 실습 3) + 실행용 자동테스트_시나리오.xlsx
증적	자동테스트결과_local.xlsx / _prod.xlsx + 스크린샷 (P-507, 실습 3·5)
파생	배포검증결과.md (P-505, 실습 5) · Slack 검수 리포트 (n8n, 실습 6)
현행화	기능명세·정보구조·API명세 .xlsx + 화면설계서 HTML/PPTX/scene.json (P-508, 실습 7)
#16	테스트결과보고서.md (P-504, 실습 8)
#17	완료보고서.md (P-208, 실습 8) — Document Chaining 17/17 완성

5주 누적: 관리 5 + 기획 7 + 구현 4 + 검수 2 + 완료 1 = 19건 산출

W5 최종 과제 · 평가 기준 (수료 조건)

제출물	평가 기준
로컬·배포 자동 테스트 결과 2종 + 스크린샷	기본(C) 60점 — 시드 + 시나리오 + 완료보고서 초안
시드 생성 + 정리 스크립트	보통(B) 80점 — 로컬·배포 각 1회 + Slack + P-505
n8n 임포트 + Slack 공유 캡처	우수(A) 100점 — 양쪽 80%+ 및 차이 분석
P-505 배포검증결과 · 현행화 산출물	+ 현행화 3종(xlsx · PPTX · Figma)
완료보고서 + .progress.md 100% 캡처	+ 완료보고서 교훈 포함

제출: GitHub 저장소 URL + NotebookLM URL + 라이브 배포 URL (교육 종료 후 1주)

W5 핵심 메시지 5가지

1. 테스트 코드 0줄 PM은 시나리오를 정의하고, /auto-scen-test가 실제 화면을 자동 테스트한다

2. 로컬 → 배포는 순서 로컬에서 룰을 잡고 배포본에서 환경 결함을 잡는다 — 릴리스 차단 결함 4건이 실재

3. 테스트는 데이터부터 시드가 없으면 Fail이 아니라 Block — 시나리오와 데이터를 함께 설계한다

4. 근거 없으면 N/A Pass Rate를 올리려 룰을 느슨하게 만들면 증적이 아니라 알리바이가 된다

5. 코드가 진실 구현하면 사양이 바뀐다 — 현행화해야 인수인계와 유지보수가 산다

5주 여정 완료 — 환경(W1) → 관리(W2) → 기획(W3) → 구현(W4) → 검수·현행화(W5)

Thank you

phk4568@naver.com

5주간 수고하셨습니다 — EDU-200에서 더 깊은 자동화로 만나요